AWS VPC PEERING

Private Connectivity Between Two VPCs

Project Report

EWB Cloud Labs – AWS Architect Associate | Project 01 | October 2026

Student name	Devesh Chak
Batch	[Enter batch]
AWS Region	[Enter actual Region, e.g. ap-south-1]
Completion date	[Enter date]
GitHub repository	[Paste repository URL]

Evidence status: This report is a detailed, workbook-based draft. The supplied PDF is an instruction workbook; it does not contain the student's actual AWS resource IDs, console screenshots, terminal test output or cleanup evidence. Fields requiring deployment evidence are clearly marked for completion. Expected outcomes are not represented as observed results.

1. Executive Summary

This project is designed to establish private IPv4 connectivity between two isolated Amazon Virtual Private Clouds (VPCs) using an AWS VPC Peering connection. VPC A hosts a public bastion and a private application instance (App-A); VPC B hosts a second private application instance (App-B). Administration originates from the student's laptop and traverses the bastion using SSH ProxyJump. Application traffic travels directly between the private workloads over VPC peering.

The design demonstrates non-overlapping CIDR planning, public/private subnet separation, explicit route-table associations, forward and return routing, least-privilege security-group rules, baseline and post-peering validation, controlled ICMP failure/recovery, and resource cleanup. The workbook defines acceptance criteria, not completed execution results; all actual outcomes must be supported by screenshots or command output.

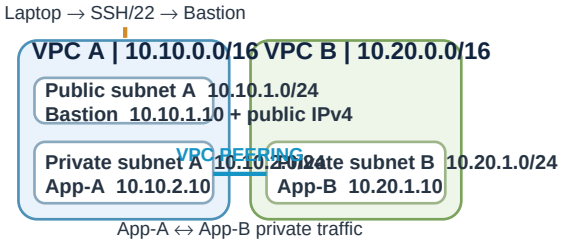
2. Scope, Requirements and Design Goals

The workbook's mission is to build a working private network connection and explain why each route and security rule is required. Both application servers must remain private and communicate using private IPv4 addresses. One public bastion in VPC A provides administration access. The lab is intended for one AWS account and one Region, using Amazon Linux 2023 and a local OpenSSH client (WSL is suitable on Windows). No NAT gateway is required.

Requirement	Implementation / proof expected
Two non-overlapping VPCs	VPC A 10.10.0.0/16; VPC B 10.20.0.0/16.
Three subnets	One public subnet in VPC A and one private subnet in each VPC.
Three explicit route tables	Public A, private A and private B, each associated with its named subnet.
Secure administration	Laptop → bastion → App-A/App-B using ProxyJump; private key remains on laptop.
Private VPC connectivity	Active VPC peering plus routes in both directions and required inbound rules.
Validation	Baseline failure, forward/reverse ping success, App-B SSH proof, and controlled ICMP failure/recovery.
Cleanup	Remove lab compute, storage, network and credential resources; check billing.

3. Architecture and Network Flow

The conceptual architecture below follows page 3 of the workbook. It is a design diagram, not an AWS-discovered topology; actual resource IDs and the deployed peering identifier must be added from the student's account.



Administration path: the laptop uses its locally stored private key to authenticate to the bastion, then ProxyJump opens a separate SSH connection to the private destination. The bastion must have a route to the destination VPC; App-B's inbound SSH rule uses the bastion's private address (10.10.1.10/32). Application traffic between App-A and App-B uses their private IPs over peering. Peering does not transitively provide VPC B with internet access through VPC A.

4. Resource Inventory and Address Plan

The following names and IPs are prescribed by the workbook (pages 4–7). These are planned values—not proof that resources with these exact addresses were launched. Add actual AWS resource IDs, actual Region and public IPv4 from the EC2/VPC console.

Resource / Name tag	VPC / subnet	CIDR or private IPv4	Actual resource ID / evidence
ewb-vpc-a	VPC A	10.10.0.0/16	[vpc-…]
ewb-a-public	VPC A	10.10.1.0/24	[subnet-…]
ewb-a-private	VPC A	10.10.2.0/24	[subnet-…]
ewb-vpc-b	VPC B	10.20.0.0/16	[vpc-…]
ewb-b-private	VPC B	10.20.1.0/24	[subnet-…]
ewb-igw-a	Attached to VPC A	Internet gateway	[igw-…]
ewb-bastion	ewb-a-public	10.10.1.10 + public IPv4	[i-…] / public IP: [fill]
ewb-app-a	ewb-a-private	10.10.2.10 / no public IP	[i-…]
ewb-app-b	ewb-b-private	10.20.1.10 / no public IP	[i-…]
ewb-a-b-peer	VPC A ↔ VPC B	Active peering	[pcx-…]

Additional identifiers to record: AWS account alias or masked account ID [fill]; Region [fill]; availability zone(s) [fill]; key pair name ewb-lab; security group IDs [fill]; route-table IDs [fill]. Do not include private-key contents, passwords, access tokens or unmasked secrets in this report.

5. Subnet CIDRs and Route Tables

The workbook specifies three route tables with explicit subnet associations (pages 5 and 10). AWS-created local routes must remain present. The peering route must be installed on both private sides for bidirectional application

traffic. The public route table also needs a route to VPC B because the bastion initiates the SSH connection to App-B.

Route table	Explicit subnet association	Destination	Target	Purpose
ewb-rt-a-public	ewb-a-public (10.10.1.0/24)	10.10.0.0/16	local	VPC A local traffic
ewb-rt-a-public	ewb-a-public	0.0.0.0/0	ewb-igw-a (igw-…)	Bastion internet access
ewb-rt-a-public	ewb-a-public	10.20.0.0/16	ewb-a-b-peer (pcx-…)	Bastion → App-B over peering
ewb-rt-a-private	ewb-a-private (10.10.2.0/24)	10.10.0.0/16	local	VPC A local traffic
ewb-rt-a-private	ewb-a-private	10.20.0.0/16	ewb-a-b-peer (pcx-…)	App-A → VPC B
ewb-rt-b-private	ewb-b-private (10.20.1.0/24)	10.20.0.0/16	local	VPC B local traffic
ewb-rt-b-private	ewb-b-private	10.10.0.0/16	ewb-a-b-peer (pcx-…)	Return path to VPC A

Subnet association evidence to attach: route-table details screenshots showing ewb-a-public associated with ewb-rt-a-public; ewb-a-private associated with ewb-rt-a-private; and ewb-b-private associated with ewb-rt-b-private. Replace pcx-… and igw-… with actual IDs.

6. VPC Peering Connection

Create a same-account, same-Region peering request named ewb-a-b-peer. Set VPC A as requester and VPC B as accepter. Accept the request and wait for Status = Active. Record the actual peering connection ID (pcx-…). VPC A CIDR is 10.10.0.0/16 and VPC B CIDR is 10.20.0.0/16; the ranges are non-overlapping. A screenshot showing Active status and both VPC IDs is required.

Actual status: [Not provided in source workbook — insert console result: Active / status reason].
Peering ID: [pcx-…]
Requester VPC ID: [vpc-…]
Accepter VPC ID: [vpc-…]

7. Security Group Inbound Rules

Security group	VPC	Inbound type / protocol	Port / ICMP type	Source
ewb-sg-bastion	VPC A	SSH / TCP	22	Current public IPv4 of student's laptop /32
ewb-sg-app-a	VPC A	SSH / TCP	22	10.10.1.10/32 (bastion)
ewb-sg-app-a	VPC A	Custom ICMP - IPv4 / Echo request	Type 8, code 0	10.20.1.10/32 (App-B)
ewb-sg-app-b	VPC B	SSH / TCP	22	10.10.1.10/32 (bastion)
ewb-sg-app-b	VPC B	Custom ICMP - IPv4 / Echo request	Type 8, code 0	10.10.2.10/32 (App-A)

The workbook recommends retaining default allow-all outbound rules for this introductory lab and default allow-all network ACLs. Security groups are stateful, so replies to permitted traffic are automatically allowed. The /32 sources narrowly restrict access to the specific lab hosts; do not open SSH or ICMP to 0.0.0.0/0. Record each security group's actual sg-… ID and attach a screenshot of its inbound rules.

8. SSH Administration Through the Bastion

The private SSH key ewb-lab.pem remains on the laptop/WSL machine and is never copied to an EC2 instance. The workbook's SSH configuration uses the bastion as a ProxyJump for both private application hosts. Use chmod 400 ~/.ssh/ewb-lab.pem and chmod 600 ~/.ssh/config.

Expected local ~/.ssh/config (replace the bastion public IPv4 with the current actual address):

```
Host ewb-bastion
HostName [BASTION_PUBLIC_IP]
User ec2-user
IdentityFile ~/.ssh/ewb-lab.pem
IdentitiesOnly yes

Host ewb-app-a
HostName 10.10.2.10
User ec2-user
IdentityFile ~/.ssh/ewb-lab.pem
IdentitiesOnly yes
ProxyJump ewb-bastion

Host ewb-app-b
HostName 10.20.1.10
User ec2-user
IdentityFile ~/.ssh/ewb-lab.pem
IdentitiesOnly yes
ProxyJump ewb-bastion
```

The workbook requires first verifying ssh ewb-bastion and ssh ewb-app-a before peering. App-B SSH is expected only after the peering and route-table changes are complete. The user's earlier troubleshooting context included a TCP/22 timeout to public IP 15.206.159.22; that is not proof that the bastion ultimately became reachable. Confirm the final current address and capture a successful App-B shell before marking this requirement complete.

9. Connectivity Validation and Test Evidence

The workbook explicitly states that its expected results are acceptance criteria, not completed results (page 11). The table below therefore distinguishes prescribed expected outcomes from evidence not present in the supplied PDF.

Test	Command / location	Workbook acceptance criterion	Actual result / evidence
Baseline failure	Inside App-A: ping -c 4 10.20.1.10, before peering and peer routes	No replies	[Attach actual output and date/time]
Forward success	Inside App-A: ping -c 4 10.20.1.10, after peering/routes/rules	4 replies; 0% packet loss	[Attach actual output]
Reverse success	Inside App-B: ping -c 4 10.10.2.10, after routes/rules	4 replies; 0% packet loss	[Attach actual output]
Private SSH proof	Laptop: ssh ewb-app-b	Shell on App-B through bastion	[Attach terminal screenshot showing hostname]
Isolation	EC2 details for App-A and App-B	No public IPv4 on either app host	[Attach EC2 details screenshot]

For each host, capture identity with hostname and hostname -I before executing the ping. This verifies which instance produced the output. Label screenshots with host, test, and result. If the first four-packet test loses a packet while the host becomes ready, wait briefly and repeat as the workbook instructs.

10. Controlled ICMP Failure and Recovery

After successful forward ping has been established, temporarily remove only the inbound Custom ICMP IPv4 Echo Request rule from ewb-sg-app-b whose source is 10.10.2.10/32. Do not remove the SSH rule. Repeat ping -c 4 10.20.1.10 from App-A; replies should stop because App-B's security group no longer permits the incoming echo requests from App-A. Restore the exact ICMP rule and repeat the ping; replies should resume. This controlled change

isolates the security-group policy as the variable while leaving peering, routes and SSH access unchanged.

Stage	Configuration / action	Evidence to attach
Before rule removal	App-B inbound ICMP Echo request from 10.10.2.10/32 is present; forward ping succeeds.	Successful ping output + SG rule screenshot.
Controlled failure	Remove only the App-B inbound ICMP Echo request rule; repeat forward ping from App-A.	Ping output showing no replies + SG rule screenshot.
Recovery	Restore the same App-B inbound ICMP rule; rerun forward ping.	Successful ping output + restored rule screenshot.

This is an intentional security-policy test, not a peering outage. The network path remains configured; packets are rejected at the destination security-group boundary. Recovery after restoring the rule demonstrates that the earlier failure was caused by the ICMP permission, not by a missing route or inactive peering.

11. Troubleshooting Notes

Symptom	Workbook's diagnostic checks
Bastion SSH times out	Check current bastion public IP; student's current public IPv4 /32 in the SSH rule; public subnet association; default 0.0.0.0/0 route to attached IGW.
App-A SSH fails	Check App-A private IP; TCP/22 inbound from 10.10.1.10/32; correct key pair, ec2-user and running instance.
App-B SSH fails	Check public route table route 10.20.0.0/16 → peering; VPC B return route 10.10.0.0/16 → peering; App-B SSH source is bastion private IP.
Ping fails after peering acceptance	Check both private subnet route tables; correct pcx-ID and Active status; Echo Request inbound rule on destination; actual host IPs.
Permission denied (publickey)	Check key pair, local IdentityFile and ec2-user. Reaching SSH authentication means routing is working.
Custom NACL failure	Check both directions, ICMP rules and SSH return ephemeral ports. The workbook assumes default allow-all ACLs.

12. Cleanup Evidence

The workbook requires saving evidence first, then removing only resources created for this project. Cleanup sequence: (1) terminate ewb-bastion, ewb-app-a and ewb-app-b; verify lab EBS volumes are deleted and remove leftover lab volumes/snapshots; release any Elastic IP; remove unused lab key pair and local lab key if no longer needed. (2) Delete peer routes and ewb-a-b-peer; wait for network interfaces to disappear, then delete lab security groups; remove subnet associations as required; delete lab subnets and custom route tables; detach and delete ewb-igw-a; delete both VPCs. (3) Filter by project tag and inspect EC2, volumes, public IPv4 allocations and VPC resources in the Region; check Billing, noting usage reporting can lag.

Cleanup item	Required proof / actual status
EC2 instances	[Screenshot/filter showing three lab instances terminated or absent]
EBS volumes and snapshots	[Evidence no lab volumes/snapshots remain]
Elastic IP / public IPv4 allocations	[Evidence any lab Elastic IP released; no unwanted allocation]
Peering and routes	[Peering deleted; lab peer routes removed]
Security groups / subnets / route tables	[Lab-only resources deleted]
Internet gateway and VPCs	[IGW detached/deleted; both VPCs deleted]
Billing review	[Billing screenshot/date; note reporting delay]

Do not claim cleanup is complete until the AWS console confirms the resources are gone. Preserve report evidence before deletion, particularly instance IDs, private IPs, peering status, route tables, security group rules and ping/SSH

outputs.

13. Learning Outcomes

This lab illustrates that VPC peering is only a network connection; it does not automatically configure reachability. Both sides need routes, and return traffic needs a valid reverse route. The public route table needs a peering route because the bastion itself opens the SSH session to App-B. Security groups enforce host-level inbound policy and can deliberately block ICMP while leaving SSH permitted. A baseline failure, two-way ping success, ProxyJump SSH proof, and controlled ICMP removal/recovery together provide stronger verification than an Active peering status alone. Peering is one-to-one and non-transitive; a peering path between A and B does not create connectivity from B to a third VPC through A.

14. Final Submission Readiness

Deliverable	Status
Cover: name, batch, project, Region, completion date	Name prefilled; remaining fields to complete
Architecture diagram and inventory with actual resource IDs/private IPs	Diagram and planned IPs included; actual IDs not supplied
Subnet CIDRs and three route tables with associations	Workbook-derived configuration included; confirm actual console evidence
Active peering and inbound rules for each SG	Expected configuration included; actual Active status and SG screenshots missing
Baseline, forward/reverse ping, App-B SSH proof	Test method and acceptance criteria included; actual outputs missing
Controlled ICMP failure and recovery	Procedure and rationale included; actual output missing
Cleanup evidence and learning note	Cleanup checklist and learning note included; cleanup proof missing

Final report note: This document is intentionally transparent about evidence gaps. Before submission, replace every bracketed placeholder and attach the student's own AWS console screenshots and terminal output. Do not substitute the workbook's expected outcomes for actual test evidence.

Source

Primary source: EWB Cloud Labs, *Student Project Workbook – AWS VPC Peering*, Project 01, October 2026, 14 pages. The architecture and addressing plan are from pages 3–4; network and security configuration from pages 5–7; ProxyJump setup from page 8; peering and routes from pages 9–10; tests and troubleshooting from pages 11–12; submission and cleanup requirements from pages 13–14.